SDP Formulation for Composite Laminates

Mostafa M. Abdalla

Abstract

We show that composite laminate design may be formulated as a non-linear semi-definite program and develop the corresponding algorithms.

1 Feasible Region of Inplane Lamination Parameters

It can be easily shown that the feasible region of inplane lamination parameters corresponds to the condition of a positive semi-definite inplane stiffness matrix in the limit of infinitely stiff fibres. Taking the limit as E_2/E_1, G_{12}/E_1 go to zero the membrane stiffness matrix of a composite laminate takes the form,

$$\boldsymbol{A}_\infty = \frac{E_1}{8}\begin{pmatrix} V_3 + 4\,V_1 + 3 & 1 - V_3 & V_4 + 2\,V_2 \\ 1 - V_3 & V_3 - 4\,V_1 + 3 & 2\,V_2 - V_4 \\ V_4 + 2\,V_2 & 2\,V_2 - V_4 & 1 - V_3 \end{pmatrix}. \tag{1}$$

To verify that the condition $\boldsymbol{A} \succeq \boldsymbol{0}$ reproduces the feasible region of lamination parameters, we set the principal determinants of the matrix to be non negative. Setting $A_{33} \geq 0$ we get,

$$V_3 \leq 1 \tag{2}$$

Setting the leading 2×2 determinant to be non-negative we get,

$$2V_1^2 - V_3 \leq 1 \tag{3}$$

Setting the determinant of the matrix to be non-negative we obtain,

$$V_4^2 + V_3^2 + 2V_2^2(1 + V_3) + 2V_1^2(1 - V_3) - 4V_1V_2V_4 \leq 1. \tag{4}$$

These equations 2-4 are identical to the well-known equations for the feasible region of inplane lamination parameters.

It is well known that the condition $\boldsymbol{A} \succeq \boldsymbol{0}$ is equivalent to $\boldsymbol{P}^T\boldsymbol{A}\boldsymbol{P} \succeq \boldsymbol{0}$ for any invertible matrix $\boldsymbol{P}$. We use this fact to simplify the representation of the matrix constraint. Let $\boldsymbol{A}_0$ be the free term of the matrix $\boldsymbol{A}_\infty$ (which corresponds to a quasi-isotropic laminate), we have,

$$\boldsymbol{A_0} = \frac{E_1}{8}\begin{pmatrix} 3 & 1 & 0 \\ 1 & 3 & 0 \\ 0 & 0 & 1 \end{pmatrix}. \tag{5}$$

Using spectral decomposition we may write $\boldsymbol{A}_0$ as $\boldsymbol{Q}^T\boldsymbol{D}\boldsymbol{Q}$ where Q is a rotation matrix and $\boldsymbol{D}$ is a diagonal matrix containing the eigenvalues ordered in

descending order. Setting $\boldsymbol{P} = \boldsymbol{Q}\boldsymbol{D}^{-1/2}$, then $\boldsymbol{A_0}^{-1} = \boldsymbol{P}\boldsymbol{P}^T$. Thus the equiavlent constraint $\boldsymbol{P}^T\boldsymbol{A}\boldsymbol{P} \succeq \boldsymbol{0}$ will have a unity matrix for a free term. Carrying out the calculations the simplified constraint takes the form,

$$\boldsymbol{X} = \begin{pmatrix} 1 & \sqrt{2}\,V_1 & \sqrt{2}\,V_2 \\ \sqrt{2}\,V_1 & 1+V_3 & V_4 \\ \sqrt{2}\,V_2 & V_4 & 1-V_3 \end{pmatrix} \succeq \boldsymbol{0}. \tag{6}$$

Note that in the transformed matrix each element depends on only one of the four lamination parameters.

To further simplify the constraint matrix let us define the following vector,

$$\boldsymbol{x} = \left(\sqrt{2}V_1, V_3, \sqrt{2}V_2, V_4\right)^t, \tag{7}$$

then the feasible region is given by $\boldsymbol{X}(\boldsymbol{x}) \succeq \boldsymbol{0}$; where,

$$\boldsymbol{X} = \begin{pmatrix} 1 & x_1 & x_3 \\ x_1 & 1+x_2 & x_4 \\ x_3 & x_4 & 1-x_2 \end{pmatrix}. \tag{8}$$

We denote the convex set of points $\boldsymbol{x}$ satisfying (8) by $\mathcal{M}$.

2 Optimisation over the Miki Cone

Consider the following optimisation problem,

$$\min_{\boldsymbol{x}\in\mathcal{M}} f(\boldsymbol{x}). \tag{9}$$

The Lagrangian for this problem has the form,

$$\mathcal{L} = f(\boldsymbol{x}) - \boldsymbol{Y}\cdot\Big(\boldsymbol{X} - \boldsymbol{Z}) - \mu \ln\det \boldsymbol{Z}, \tag{10}$$

where $\boldsymbol{Y} \succeq \boldsymbol{0}$ is a Lagrange multiplier and $\boldsymbol{Z} \succeq \boldsymbol{0}$ is a slack variable. The final term is the logarithmic barrier interior penalty term. The exact solution is recovered in the limit $\mu \to 0^+$.

Setting the derivatives of the Lagrangian to zero with respect to all variables we obtain the first order optimality conditions,

$$\boldsymbol{Y}: \qquad \boldsymbol{X} - \boldsymbol{Z} = \boldsymbol{0}, \tag{11}$$

$$\boldsymbol{Z}: \qquad \boldsymbol{Y} - \mu\boldsymbol{Z}^{-1} = \boldsymbol{0}, \tag{12}$$

$$\boldsymbol{x}: \qquad \boldsymbol{g} - \boldsymbol{\mathcal{M}}^t\cdot\boldsymbol{Y} = \boldsymbol{0}, \tag{13}$$

where $\boldsymbol{g} = \partial_{\boldsymbol{x}} f$ is the gradient of the objective function and $\boldsymbol{\mathcal{M}} = \partial_{\boldsymbol{x}}\boldsymbol{X}$ is the *constant* gradient of the matrix $\boldsymbol{X}$ in (8). The components of third order tensor $\boldsymbol{\mathcal{M}}$ are defined as,

$$\mathcal{M}_{ijm} = \frac{\partial X_{ij}}{\partial x_m}. \tag{14}$$

Two types of multiplication by a matrix are defined, note the use of the summation convention,

$$\left(\boldsymbol{\mathcal{M}}^t\boldsymbol{A}\right)_{ijm} = \mathcal{M}_{ikm}A_{kj}, \tag{15}$$

and,

$$\Big(\boldsymbol{\mathcal{M}} \cdot \boldsymbol{a}\Big)_{ij} = \mathcal{M}_{ijm} a_m, \tag{16}$$

$$\Big(\boldsymbol{\mathcal{M}}^t \cdot \boldsymbol{A}\Big)_{a} = \mathcal{M}_{ija} A_{ij}, \tag{17}$$

The only inner product of third order tensors we need is the following,

$$\Big(\boldsymbol{\mathcal{M}} \cdot \boldsymbol{\mathcal{N}}\Big)_{mn} = \mathcal{M}_{ijm} \mathcal{N}_{ijn}. \tag{18}$$

Equation (11) can be enforced as a strict equality eliminating $\boldsymbol{Z}$ from the set of unkowns. The consequences of this elimination will be discussed at a later point. Equation (12) is rewritten in the following form,

$$\boldsymbol{X}\boldsymbol{Y} - \mu \boldsymbol{I} = \boldsymbol{0}, \tag{19}$$

The optimal solution is obtained through solving equations (19) and (13) for $\boldsymbol{x}$ and $\boldsymbol{Y}$ while gradually relaxing μ to zero. This nonlinear system is solved using a composite Newton method with centering. Define,

$$\boldsymbol{R_Z} = \mu \boldsymbol{I} - \boldsymbol{X}\boldsymbol{Y} - d\boldsymbol{X} d\boldsymbol{Y}, \tag{20}$$

$$\boldsymbol{r_x} = \boldsymbol{\mathcal{M}}^t \cdot \boldsymbol{Y} - \boldsymbol{g}. \tag{21}$$

The Newton system is given by,

$$\boldsymbol{X} d\boldsymbol{Y} + d\boldsymbol{X}\boldsymbol{Y} = \boldsymbol{R_Z}, \tag{22}$$

$$\boldsymbol{H} d\boldsymbol{x} - \boldsymbol{\mathcal{M}}^t \cdot d\boldsymbol{Y} = \boldsymbol{r_x} \tag{23}$$

where $\boldsymbol{H} = \partial_{\boldsymbol{x}} \partial_{\boldsymbol{x}} f$ is the Hessian of the objective function. Equation (22) is used to eliminate $d\boldsymbol{Y}$,

$$d\boldsymbol{Y} = \langle \boldsymbol{X}^{-1} \big(\boldsymbol{R_Z} - d\boldsymbol{X}\boldsymbol{Y}\big) \rangle, \tag{24}$$

where $\langle \cdot \rangle$ is the symmetric part of a matrix.

Substituting equation (24) into (23) is equivalent to taking the Schur complement of the system and leads to the following convenient equation for the update of dx,

$$\big(\boldsymbol{H} + \boldsymbol{B}\big) d\boldsymbol{x} = \boldsymbol{r}'_{\boldsymbol{x}}, \tag{25}$$

where the matrix $\boldsymbol{B}$, encapsulating the penalisation introduced by the barrier function, is given by,

$$\boldsymbol{B} = \big(\boldsymbol{\mathcal{M}}^t \boldsymbol{X}^{-1}\big) \cdot \big(\boldsymbol{\mathcal{M}}^t \boldsymbol{Y}\big). \tag{26}$$

The details of the numerical procedure are listed in Algorithm (1), while the details of the internal functions and procedures are listed in Algorithm 2. The parameters β_p, β_c, and δ are empirical factors. The value of β_p controls the amount of reduction of duality gap attempted at the predictor phase of the composite Newton iteration. The value of β_c is a safety upper bound on the amount of reduction of the duality gap attempted in the corrector phase of the iteration. The value of δ controls how close to the boundary of the semi-definite cone the algorithm is allowed to step. Typical values are $\beta_p = 0$, $\beta_c = 0.1$, and $\delta = 0.05$.

Note that the current algorithm needs to start at a feasible design vector $\boldsymbol{x}$ such that $\boldsymbol{X} \succeq \boldsymbol{0}$. The step size in the composite Newton iteration is chosen to maintain positive definite primal, $\boldsymbol{X}$, and dual, $\boldsymbol{Y}$, matrices throughout the iterations.

Algorithm 1 optimisation over the Miki cone

```
μ ← f̄                                   ▷ f̄ is the scale of the objective function
x ← 0
X ← I, Y ← μI, dX, dY ← 0
Δ ← DualityGap(X, dX, Y, dY)
while Δ > tol do
    ObjectiveFunction(x, f, g, H)
    μ ← Δ/3
    r ← −g
    CalculateResiduals(β_p μ, r, R)
    CalculateHessian(H, X, Y)
    dx ← H^{−1} dr
    UpdateIncrements(dx, X, dX, Y, dY, R_Z)
    Δ̃ ← DualityGap(X, dX, Y, dY)
    β̃_c ← CorrectorMultiplier(Δ, Δ̃)
    r ← −g
    CalculateResiduals(β̃_c μ, r_x, R_Z)
    dx ← H^{−1} dr
    UpdateIncrements(dx, X, dX, Y, dY, R_Z)
    α_p ← StepSize(X, dX)
    α_d ← StepSize(Y, dY)
    x ← x + α_p dx
    UpdateVariables(α_p, α_d, X, dX, Y, dY)
    Δ ← DualityGap(X, dX, Y, dY)
end while
```

Algorithm 2 internal procedures

function DualityGap($\boldsymbol{X}, d\boldsymbol{X}, \boldsymbol{Y}, d\boldsymbol{Y}$)
  return $\big(\boldsymbol{X} + d\boldsymbol{X}\big) \cdot \big(\boldsymbol{Y} + d\boldsymbol{Y}\big)$
end function

procedure CalculateResiduals($\mu, \boldsymbol{r}, \boldsymbol{R_Z}$) ▷ residual calculations
  $\boldsymbol{R_Z} \leftarrow \mu \boldsymbol{I} - \boldsymbol{X}\boldsymbol{Y} - d\boldsymbol{X} d\boldsymbol{Y}$
  $\boldsymbol{r} \leftarrow \boldsymbol{r}\boldsymbol{\mathcal{M}}^t \cdot \boldsymbol{Y}$
  $d\boldsymbol{Y} \leftarrow \boldsymbol{X}^{-1}\boldsymbol{R_Z}$
  $\boldsymbol{r} \leftarrow \boldsymbol{r} + \boldsymbol{\mathcal{M}}^t \cdot d\boldsymbol{Y}$
end procedure

procedure UpdateIncrements($\boldsymbol{dx}, \boldsymbol{X}, d\boldsymbol{X}, \boldsymbol{Y}, d\boldsymbol{Y}, \boldsymbol{R_Z}$)
  $d\boldsymbol{X} \leftarrow \boldsymbol{\mathcal{M}} d\boldsymbol{x}$
  $d\boldsymbol{Y} \leftarrow \langle \boldsymbol{X}^{-1}\big(\boldsymbol{R_Z} - d\boldsymbol{X}\boldsymbol{Y}\big)\rangle$
end procedure

procedure CalculateHessian($\boldsymbol{H}, \boldsymbol{X}, \boldsymbol{Y}$)
  $\boldsymbol{H} \leftarrow \boldsymbol{H} + \big(\boldsymbol{\mathcal{M}}^t \boldsymbol{X}^{-1}\big) \cdot \big(\boldsymbol{\mathcal{M}}^t \boldsymbol{Y}\big)$
end procedure

function CorrectorMultiplier($\Delta, \tilde{\Delta}$)
  $\rho \leftarrow \dfrac{\min\{\Delta, \tilde{\Delta}\}}{\Delta}$
  return $\max\{\rho^2, \beta_c\}$
end function

function StepSize($\boldsymbol{A}, d\boldsymbol{A}$)
  $\lambda \leftarrow \min\{\lambda \,|\, \exists \boldsymbol{a} \neq \boldsymbol{0}, \big(d\boldsymbol{A} - \lambda \boldsymbol{A}\big)\boldsymbol{a} = \boldsymbol{0}\}$
  $\alpha \leftarrow \max\{-\lambda, 1 - \delta\}$
  return $\dfrac{1 - \delta}{\alpha}$
end function

procedure UpdateVariables($\alpha_p, \alpha_d, \boldsymbol{X}, d\boldsymbol{X}, \boldsymbol{Y}, d\boldsymbol{Y}$)
  $\boldsymbol{X} \leftarrow \boldsymbol{X} + \alpha_p d\boldsymbol{X}$
  $d\boldsymbol{X} \leftarrow \boldsymbol{0}$
  $\boldsymbol{Y} \leftarrow \boldsymbol{Y} + \alpha_d d\boldsymbol{Y}$
  $d\boldsymbol{Y} \leftarrow \boldsymbol{0}$
end procedure

3 General Loading

When the laminate is subject to a combination of inplane and bending loads, it is no longer sufficient to consider inplane lamination parameters. We follow a similar definition of lamination parameters to that in Hammer et al,

$$\boldsymbol{v}_\alpha = \frac{\alpha+1}{2}\int_{-1}^{1} \zeta^\alpha \boldsymbol{x}(\zeta) d\zeta, \tag{27}$$

where $\boldsymbol{x}(\zeta) \in \mathcal{M}$, and $\zeta = 2z/h$ is a nondimensional coordinate in the thickness direction.

In the general case of non-symmetric laminates $\alpha = 0, 1, 2$ correspond to inplane, coupling, and bending lamination parameters respectively. For symmetric laminates only $\alpha = 0$and 2 need be considered and the integrals may be restricted to the range $\zeta \in [0, 1]$ and multiplied by 2.

The feasible region for lamination parameters may be approximated to arbitrary accuracy by discretising the integrals using the mid-point rule. First, partition the interval $[0, 1]$ (or $[-1, 1]$ for the non-symmetric case) using a set of abscissa $0 < z_1 < z_2 < \cdots < z_{n-1} < 1$, then,

$$\boldsymbol{v}_\alpha = \sum_{\ell=1}^{n} w_\ell^\alpha \boldsymbol{x}_\ell, \tag{28}$$

where $\boldsymbol{x}_\ell \in \mathcal{M}$, $\ell = 1, 2, \ldots, n$.

We think of each interval as a *sublaminate*. The total number of sublaminates is n_s. The sublaminate weights $w_{\alpha\ell}$ are given by,

$$w_{\alpha\ell} = \frac{1}{2}\left(z_{\ell+1}^{\alpha+1} - z_\ell^{\alpha+1}\right). \tag{29}$$

Thus the feasible region of lamination parameters under combined loading can be approximated by the projection, as per (28), of the cartesian product of n_s Miki cones onto the space of lamination parameters. Now, the optimisation problem over lamination parameters may be solved by an extension of the method discussed in sec. 2.

4 Optimisation over Lamination Parameters

The model problem we use is the following,

$$\begin{aligned} &\min_{\boldsymbol{v}} f(\boldsymbol{v}) \\ &\text{subject to:} \\ &\qquad \boldsymbol{v}_\alpha - \sum_{\ell=1}^{n} w_{\alpha\ell}\boldsymbol{x}_\ell = \boldsymbol{0} \\ &\qquad \boldsymbol{X}(\boldsymbol{x}_\ell) \succeq \boldsymbol{0} \end{aligned} \tag{30}$$

Forming the Lagrangian we have,

$$\mathcal{L} = f + \sum_\alpha \boldsymbol{y}_\alpha^t \cdot \left(\boldsymbol{v}_\alpha - \sum_\ell w_{\alpha\ell} \boldsymbol{x}_\ell \right) - \sum_\ell \boldsymbol{Y}_\ell \cdot \left(\boldsymbol{X}_\ell - \boldsymbol{Z}_\ell \right) - \mu \sum_\ell \ln \det \boldsymbol{Z}_\ell \quad (31)$$

where $\boldsymbol{y}_\alpha$ are Lagrange multipliers associated with the projection constraint, and $\boldsymbol{Y}_\ell$ and $\boldsymbol{Z}_\ell$ are as before.

The optimality conditions are obtained by setting the derivatives of the Lagrangian with respect to all variables to zero. After some manipulation similar to sec. 2, we get,

$$\boldsymbol{X}_\ell \cdot \boldsymbol{Y}_\ell - \mu \boldsymbol{I} = \mathbf{0}, \quad (32)$$

$$-\boldsymbol{\mathcal{M}}^t \cdot \boldsymbol{Y}_\ell - \sum_\alpha w_{\alpha\ell} \boldsymbol{y}_\alpha = \mathbf{0}, \quad (33)$$

$$\boldsymbol{v}_\alpha - \sum_\ell w_{\alpha\ell} \boldsymbol{x}_\ell = \mathbf{0}, \quad (34)$$

$$\boldsymbol{g}_\alpha + \boldsymbol{y}_\alpha = \mathbf{0}. \quad (35)$$

Linearisation leads to the following set of equations,

$$d\boldsymbol{X}_\ell \cdot \boldsymbol{Y}_\ell + \boldsymbol{X}_\ell \cdot d\boldsymbol{Y}_\ell = \boldsymbol{R}_{\boldsymbol{Z}_\ell}, \quad (36)$$

$$-\boldsymbol{\mathcal{M}}^t \cdot d\boldsymbol{Y}_\ell - \sum_\alpha w_{\alpha\ell} d\boldsymbol{y}_\alpha = \boldsymbol{r}_{\boldsymbol{x}_\ell}, \quad (37)$$

$$d\boldsymbol{v}_\alpha - \sum_\ell w_{\alpha\ell} d\boldsymbol{x}_\ell = \boldsymbol{r}_{\boldsymbol{y}_\alpha}, \quad (38)$$

$$\sum_\beta \boldsymbol{H}_{\alpha\beta} d\boldsymbol{v}_\beta + d\boldsymbol{y}_\alpha = \mathbf{0} \quad (39)$$

Here $\boldsymbol{H}_{\alpha\beta} = \partial_{\boldsymbol{V}_\alpha} \partial_{\boldsymbol{V}_\beta} f$. Note that we use,

$$\boldsymbol{y}_\alpha = d\boldsymbol{y}_\alpha. \quad (40)$$

The Lagrange multipliers associated with the projection (28) are reset according to (40). The increments $d\boldsymbol{y}_\alpha$ are used only as an intermediate variable for the calculation of consistent increments of other variables.

Eliminating $d\boldsymbol{Y}_\ell$ leads to the following reduced system,

$$\boldsymbol{B}_\ell \cdot d\boldsymbol{x}_\ell - \sum_\alpha w_{\alpha\ell} d\boldsymbol{y}_\alpha = \boldsymbol{r}'_{\boldsymbol{x}_\ell}, \quad (41)$$

$$d\boldsymbol{v}_\alpha - \sum_\ell w_{\alpha\ell} d\boldsymbol{x}_\ell = \boldsymbol{r}_{\boldsymbol{y}_\alpha}, \quad (42)$$

$$\sum_\beta \boldsymbol{H}_{\alpha\beta} d\boldsymbol{v}_\beta + d\boldsymbol{y}_\alpha = \boldsymbol{r}_{\boldsymbol{v}_\alpha}. \quad (43)$$

Algorithm 3 internal procedures

```
function DualityGap( )
    gap ← 0
    for ℓ = 1 ... n_s do
        gap ← gap + S_ℓ.DualityGap( )
    end for
    return gap
end function
```

```
procedure CalculateResiduals(μ, v, r_v)
    r_y ← v
    for ℓ = 1 ... n_s do
        r_{x_ℓ} ← 0
        S_ℓ.CalculateResiduals(μ, x_ℓ, r_{x_ℓ})
        for α = 1 ... n_p do
            r_{y_α} ← r_{y_α} + w_{αℓ} x_ℓ
        end for
        B_ℓ ← S_ℓ.B_ℓ
        for α = 1 ... n_p do
            r_{y_α} ← r_{y_α} − w_{αℓ} B_ℓ^{-1} r_{x_ℓ}
        end for
    end for
    r_v ← r_v − Q^{-1} r_y
end procedure
```

```
procedure UpdateIncrements(dv)
    r_y ← r_y + dv
    dy ← Q^{-1} r_y
    for ℓ = 1 ... n_s do
        for α = 1 ... n_p do
            r_{x_α} ← r_{x_α} + w_{αℓ} dy_α
        end for
        dx_ℓ ← B_ℓ^{-1} r_{x_ℓ}
        S_ℓ.UpdateIncrements(dx_ℓ)
    end for
end procedure
```

```
procedure CalculateHessian(H)
    Q ← 0
    for ℓ = 1 ... n_s do
        B_ℓ ← 0
        S_ℓ.CalculateHessian(B_ℓ)
        for α = 1 ... n_p do
            for β = 1 ... n_p do
                Q_{αβ} ← Q_{αβ} + w_{αℓ} w_{βℓ} B_ℓ^{-1}
            end for
        end for
    end for
    H ← H + Q^{-1}
end procedure
```

```
function StepSize(α_p, α_d)
    S_1.StepSize(α_p, α_d)
    for ℓ = 2 ... n_s do
        S_ℓ.StepSize(α_{pℓ}, α_{dℓ})
        α_p ← min{α_p, α_{pℓ}}
        α_d ← min{α_d, α_{dℓ}}
    end for
end function
```

```
procedure UpdateVariables(α_p, α_d, X, dX, Y, dY)
    v ← v + α_p dv
    for ℓ = 1 ... n_s do
        S_ℓ.UpdateVariables(α_p, α_d)
    end for
end procedure
```